

Course Outline

- **Lecture 1: Basic Linear Algebra**
 - Data Structures, Vectors, and Matrices
 - Vector Operations and Vector Spaces
 - Vector Norms
 - K-Nearest Neighbors (KNN)
 - Dot Products
 - Linear Models
(binary classification, KNN & Decision Plane)
- **Lecture 2: Transformations & Decompositions**
 - Matrix Multiplication
 - Ordinary Least Squares (OLS)
 - Matrix Transformations
 - Basis, Change of Basis
 - Eigen Algebra
 - Principal Components Regression
(regression, OLS & PC regression)
- **Lecture 3: Probability and Statistics**
 - Hypothesis Testing
 - Random Variables
 - Probability Distributions (discrete, continuous)
(hypothesis testing, AB testing)
- **Lecture 4: Differential Calculus**
 - Derivatives
 - Gradient Descent
 - Multivariate Testing
 - Automatic Differentiation
(binary classification, Logistic Regression)
- **Lecture 5: Bayes & Information Theory**
 - Fraud Detection
 - Bayes Theorem
 - Naïve Bayes
 - Information Theory
(binary classification, Naïve Bayes)
- **Lecture 6: Final Project Presentations**
(Recommender System)

En este curso se recorren dos grupos de contenidos:

- Conceptos de Álgebra Lineal, Cálculo diferencial, Probabilidad y estadística
- Métodos predictivos basados en cálculo analítico de álgebra lineal sobre matrices (OLS, PCR), cálculo diferencial (MLE) y teoría de probabilidades (Naive Bayes)

ARITMÉTICA --> RAMA DE LAS MATEMÁTICAS QUE ESTUDIA LOS NÚMEROS Y SUS OPERACIONES

ÁLGEBRA --> RAMA DE LAS MATEMÁTICAS QUE ESTUDIA NÚMEROS Y SIMBOLOS ABSTRACTOS (ECUACIONES CON VARIABLES)

ÁLGEBRA LINEAL --> RAMA DE LAS MATEMÁTICAS QUE ESTUDIA ESPACIOS VECTORIALES y TRANSFORMACIONES LINEALES

CÁLCULO --> RAMA DE LAS MATEMÁTICAS QUE ESTUDIA CÓMO CAMBIAN LAS ECUACIONES INCREMENTANDO EL VALOR DE SUS VARIABLES EN UN RANGO ESPECÍFICO COMO ÁREA, VOLUMEN, RECORRIDO, etc..

CÁLCULO DIFERENCIAL --> cómo cambia respecto a la derivada

CÁLCULO INTEGRAL --> cómo cambia respecto a la integral

CÁLCULO INFINITESIMAL --> cómo cambia respecto al infinito

- ¿Qué es que converja un modelo? ¿Por qué puede un modelo o no converger?

Al buscar Maximizar la función de probabilidad con el método de Descent Gradient sobre LOSS function no encontrar el punto mínimo de la curva ya que las tangentes que vamos encontrando para el cálculo de los pesos de la función de estimación no convergen en ese punto mínimo (vamos saltando a ambos lados sin conseguir encontrarla).

La convergencia se alcanza cuando el error de la red neuronal, medido por la función de pérdida, deja de disminuir significativamente con respecto al número de iteraciones (o épocas), lo que indica que los parámetros de la red ya no mejoran.

La convergencia es un concepto importante en el aprendizaje porque el proceso de entrenamiento puede llevar mucho tiempo y no siempre queda claro si la red ha aprendido lo suficiente. Sin embargo, se puede decir que una red neuronal ha convergido cuando el error de entrenamiento y validación deja de disminuir. La convergencia no siempre garantiza una solución óptima, esto depende de muchos factores, como la calidad de los datos, la arquitectura de la red y los hiperparámetros utilizados. Un modelo puede converger hacia un mínimo local o un punto de equilibrio en lugar de un mínimo global, lo que se traduciría en un rendimiento inferior al óptimo.

La convergencia no siempre significa que la red haya encontrado el mínimo global de la función de pérdida, sino que podría quedarse estancada en un mínimo local. Por lo tanto, es importante supervisar las pérdidas de entrenamiento y validación y asegurarse de que no se trata de un sobreentrenamiento o insuficiente. Además, un modelo puede sobreajustarse a los datos de entrenamiento y tener un buen rendimiento en el conjunto de entrenamiento, pero mal con datos no previamente entrenados. Algunos algoritmos de optimización pueden dar con una solución deficiente debido a una inicialización deficiente, a la elección de hiperparámetros o a la falta de convergencia de la función de pérdida. Para mitigar estos problemas, se pueden utilizar técnicas como la regularización, el aumento de datos y early stopping para evitar el sobreajuste y mejorar las capacidades de generalización del modelo.

- Qué significa que un modelo sea o no paramétrico y cuáles son sus implicaciones prácticas?
Un modelo no paramétrico (KNN por ejemplo) necesita pasar por todos los puntos de sus datos para hacer las predicciones, por lo que no aprende nada y por tanto no se pueden "parametrizar" sus ejecuciones anteriores (encontrar una función con parámetros).

- ¿Qué diferencia hay entre weights y bias?
La red neuronal debe usar los pesos según sea necesario. En ocasiones, una red neuronal puede estar sobreentrenada y producir pesos grandes e incontrolables que saturan las señales neuronales. Este desorden hace que el modelo sea demasiado complejo y conduce a un concepto de aprendizaje automático llamado sobreajuste. El sobreajuste capta datos innecesarios (o ruido) que complican la predicción de salida del modelo. Cuando se produzca un sobreajuste, será necesario implementar un método de regularización del peso. La regularización reduce el peso de las conexiones al realizar actualizaciones de los algoritmos de aprendizaje. Estas actualizaciones estabilizan la generalización del modelo, es decir, su capacidad para adaptarse a los nuevos datos.

Los sesgos son funciones adicionales destinadas a ayudar a especificar el aspecto que debe tener una respuesta correcta. Solo puede haber un sesgo por capa de neuronas.

- Cómo se diferencia una columna más de una matriz de una dimensión adicional que lo convierte en un tensor de dimensión 3?

La diferencia es un valor adicional a uno de sus índices (nuevo valor de variable) frente a tener un índice adicional (nuevo valor de la relación entre las variables dimensionadas). Las operaciones que se pueden hacer sobre ellos son distintas.

Vector -- Un solo índice A_i (valor máximo de "i" dimensión del vector)

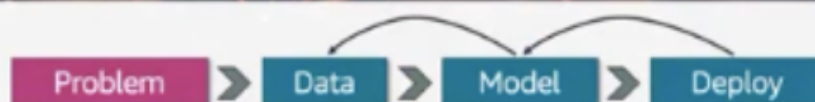
Matriz -- Dos índices $A_{i,j}$ (valores máximos de "i,j" dimensión de la matriz)

Tensor -- Tres o más índices $A_{i,j,k}$ (valores máximos de "i,j,k" dimensión del Tensor)

Having completed this class, you should be able to:

- Understand the **fundamental math and statistics** used in machine learning
- Evaluate mathematical/statistical **approaches** to find the best way to **solve** different machine learning problems
- Use linear **algebra**, differential **calculus**, **probability** and **statistics** to solve machine learning problems
- Understand the essential mathematics of **gradient descent**

Math & Machine Learning



Topics in this course	Data	Model	Deploy
Basic Linear Algebra	Representing, Transforming, Operations on data at scale	KNN, Linear Classifiers	Compute big data faster
Intermediate Linear Algebra		PCA, <u>SVD</u> , Recommendation Engines	
Probability & Calculus	Randomness in data	Regression, Neural Networks, Gradient	